CORTEXGRID : INVOICE INTELLIGENCE PLATFORM

A dissertation submitted to the Jawaharlal Nehru Technological University, Hyderabad in partial fulfillment of the requirement for the award of degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

By

CH ADITHYA ABHIRAM (23B81A0568)

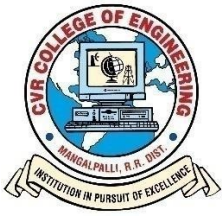
A MAHITHA (23B81A0585)

N TRISHA (23B81A05C0)

Under the guidance of

M Sunitha

Asst. Professor



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CVR COLLEGE OF ENGINEERING

(*An Autonomous institution, NBA, NAAC Accredited and Affiliated to JNTUH, Hyderabad*)

Vastunagar, Mangalpalli (V), Ibrahimpatnam (M),
Rangareddy (D), Telangana- 501 510

2026-2027

CVR COLLEGE OF ENGINEERING

(An UGC Autonomous Institution, Affiliated to JNTUH, Accredited by NBA, and NAAC)
Vastunagar, Mangalpalli (V), Ibrahimpatnam (M), Ranga Reddy (Dist.) - 501510, Telangana State.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Project Work Stage-I entitled **CORTEXGRID: INVOICE INTELLIGENCE PLATFORM** is submitted by **CH ADITHYA ABHIRAM (23B81A0568), A MAHITHA (23B81A0585), and N TRISHA (23B81A05C0)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering** for the academic year 2026–2027.

Project Supervisor
M Sunitha
Asst. Professor

Professor in-charge project
Dr. S. Suguna Mallika
Professor

Professor and Head, CSE
Dr. A. Vani Vathsala

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful companies of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance has been a source of inspiration throughout the course of the project.

It is a great pleasure to convey our profound sense of gratitude to our principal **Dr. K. Ramamohan Reddy**, Vice-Principal **Prof. L. C. Siva Reddy , Dr. A. Vani Vathsala**, Head of CSE Department, CVR College of Engineering, for having been kind enough to arrange necessary facilities for executing the project in the college.

We deem it a pleasure to acknowledge our sense of gratitude to our project guide **M Sunitha** under whom we have carried out the project work. His incisive and objective guidance and timely advice encouraged us with constant flow of energy to continue the work.

We wish a deep sense of gratitude and heartfelt thanks to management for providing excellent lab facilities and tools. Finally, we thank all those whose guidance helped us in this regard.

We place highest regards to our **Parents**, our **Friends** and **Well-wishers** who helped a lot in making the report of this project.

CH ADITHYA ABHIRAM (23B81A0568)
A MAHITHA (23B81A0585)
N TRISHA (23B8105C0)

ABSTRACT

In many organizations, finance teams receive hundreds of vendor invoices every day in the form of PDF documents, making manual verification a time-consuming and error-prone process. Employees must compare invoice details with purchase records, identify duplicate invoices, verify vendor information, and ensure that the billed amount matches the agreed price before approving payments. This project proposes **CortexGrid: Invoice Intelligence Platform**, an intelligent web-based system that automates invoice processing, verification, and fraud detection. The platform extracts important information from uploaded invoice PDFs, validates the extracted data against purchase records, detects duplicate invoices, identifies price mismatches, and performs intelligent vendor name matching to improve verification accuracy. To efficiently handle large volumes of invoices, the system processes uploaded documents asynchronously in the background, ensuring a smooth user experience without affecting application performance. Based on the verification results, the platform generates a risk score for each invoice and presents the findings through a user-friendly dashboard, enabling finance teams to quickly identify suspicious transactions and make informed decisions. The system is developed using a full-stack architecture with a responsive web interface, secure backend services, background task processing, and a relational database for reliable data management. By reducing manual effort, minimizing financial errors, and improving the efficiency of invoice verification, CortexGrid provides a practical solution for organizations seeking to modernize their accounts payable process and strengthen financial control.

Table of Contents			
			Page No.
		List of Figures	6
1		**Introduction**	7
	1.1	Motivation	7
	1.2	Problem statement	8
	1.3	Project Objectives	9
	1.4	Project report Organization	10
2		**Literature Survey**	12
	2.1	Existing work	14
	2.2	Limitations of Existing work	14
3		**Software & Hardware specifications**	16
	3.1	Software requirements	16
	3.2	Functional Requirements	16
	3.3	Non-Functional Requirements	20
	3.4	Hardware requirements	21
4		**Proposed System Design**	22
	4.1	Proposed System	22
	4.2	Class Diagram	26
	4.3	Use case Diagram	28
	4.4	Activity Diagram	29
	4.5	Sequence Diagram	31
	4.6	System Architecture	32
	4.7	Technology Description	34
5		**Implementation & Testing**	-----
	5.1	Front page Screenshot	
	5.2	---	
	5.3	---	
6		**Conclusion & Future scope**	-
		References	37
		Appendix: (If any like Published paper / source code)	-

LIST OF FIGURES:

Table	Title	Page-No
4.2	Class Diagram	27
4.3	Use case Diagram	29
4.4	User Activity Diagram	30
4.5	User Sequence Diagram	32
4.6	System Architecture	33

CHAPTER-1

INTRODUCTION

1.1 MOTIVATION

In many organizations, finance teams receive a large number of vendors invoices every day, often in the form of PDF documents. Verifying these invoices manually requires employees to read the invoice details, check vendor information, compare the billed prices and quantities with purchase records, and identify whether the same invoice has already been submitted. As the number of invoices increases, this process becomes time-consuming and prone to human errors.

Invoices may also contain information in different formats, and some documents may require text extraction or OCR before their details can be processed. Differences in vendor names and invoice formats make manual verification even more difficult. These challenges can lead to incorrect payments, duplicate payments, and missed price discrepancies.

Therefore, there is a need for a system that can automate the initial processing and verification of invoices while reducing the manual effort required from finance teams. **CortexGrid: Invoice Intelligence Platform** is proposed to address these challenges by extracting relevant invoice information, processing documents in the background, and providing structured information that can be used for further verification.

In addition to automating document processing, modern Artificial Intelligence and Machine Learning techniques can be used to analyse invoice information and identify patterns that may not be easily detected through rule-based verification alone. Natural Language Processing can assist in understanding variations in vendor names and invoice text, while AI/ML-based analysis can support anomaly detection, fraud analysis, and risk assessment. Integrating these capabilities with the invoice processing pipeline can provide finance teams with more intelligent and data-driven verification support.

The system is designed to provide a practical and efficient approach to invoice processing by combining document processing, backend automation, database management, Artificial Intelligence, Machine Learning, Natural Language Processing, and intelligent verification. By reducing repetitive manual work and organizing invoice information in a centralized platform, CortexGrid aims to improve the efficiency

and reliability of the invoice verification process.

1.2 PROBLEM STATEMENT

In many organizations, finance teams receive a large number of vendor invoices in the form of PDF documents. The manual process of reading these invoices, entering their details, checking vendor information, comparing prices with purchase records, and identifying duplicate submissions requires considerable time and effort. As the number of invoices increases, manual verification can become inefficient and may result in errors or missed discrepancies.

The problem becomes more difficult when invoices have different formats or contain scanned documents that require text extraction. Vendor names may also appear in different forms across invoices, making it difficult to identify the same vendor consistently. These issues can result in duplicate payments, incorrect billing, and difficulty in identifying suspicious transactions.

The objective of **CortexGrid: Invoice Intelligence Platform** is to develop a web-based system that automates the initial invoice processing and verification process. The system should extract important information from invoice PDFs, process uploaded documents efficiently in the background, and organize the extracted information for further verification. It should provide a centralized interface through which finance teams can upload invoices, monitor their processing status, and review the extracted information.

Traditional rule-based verification methods may identify straightforward issues such as exact duplicate invoice numbers or direct price mismatches, but they may have difficulty handling variations in vendor names, inconsistent invoice formats, and complex relationships between invoice attributes. AI and Machine Learning techniques can provide additional intelligence by analyzing invoice data, identifying unusual patterns, supporting semantic vendor matching, detecting potential anomalies, and estimating the risk associated with an invoice. Therefore, an integrated system should combine conventional invoice processing and verification with AI/ML-based analysis.

The proposed system aims to reduce repetitive manual work, minimize errors, improve invoice processing efficiency, and provide intelligent verification support. By combining backend-based document processing with AI/ML, NLP, and data-driven analysis, CortexGrid provides a foundation for identifying duplicate invoices, price mismatches, vendor inconsistencies, anomalous patterns, and potentially fraudulent transactions while generating an interpretable risk assessment.

1.3 PROJECT OBJECTIVES

The objectives of the project are as follows:

- The system should provide a web-based platform for efficient invoice processing and verification.
- It should allow users to upload vendor invoices in PDF format.
- The system should extract important information such as invoice number, vendor name, invoice date, amount, and other relevant details from uploaded invoices.
- The application should support text extraction from normal PDF documents and OCR-based extraction for scanned invoices.
- The system should process uploaded invoices asynchronously in the background so that large or multiple files do not block the main application.
- The system should store invoice information securely in a relational database for efficient retrieval and management.
- The application should allow users to view uploaded invoices, their processing status, and extracted information through a user-friendly dashboard.
- The system should provide a foundation for matching invoice information with vendor and purchase-order records.
- The system should support identification of duplicate invoices and price or quantity mismatches during the verification process.
- The application should provide intelligent vendor name matching to handle variations in vendor names.
- The system should use AI/ML-based analysis to identify unusual patterns and anomalies in invoice data.
- The system should support AI-assisted fraud analysis by identifying invoice characteristics that may indicate potentially suspicious transactions.
- The system should use Natural Language Processing techniques to handle variations in vendor names and relevant invoice text.
- The system should generate a risk score and risk level by combining invoice verification results with AI/ML-based analysis and provide clear reasons for flagged invoices.
- The system should provide data-driven insights that assist finance users in reviewing potentially high-risk invoices.
- The system should reduce manual effort, minimize errors, and improve the efficiency of invoice verification and financial control.

1.4 PROJECT REPORT ORGANIZATION

Chapter 1: Introduction

This chapter introduces **CortexGrid: Invoice Intelligence Platform** and provides an overview of the proposed system. It explains the motivation for developing the system, the problem statement, project objectives, and the overall organization of the project report.

Chapter 2: Literature Survey

This chapter presents a review of existing approaches and research related to invoice processing, document intelligence, PDF text extraction, OCR, Natural Language Processing, AI/ML-based invoice analysis, anomaly detection, fraud detection, vendor matching, and automated invoice verification. It discusses existing methods and their limitations and identifies the need for an integrated invoice intelligence platform combining backend automation with AI/ML-based analysis.

Chapter 3: Software & Hardware Specifications

This chapter describes the software and hardware requirements needed for implementing CortexGrid. It also explains the functional and non-functional requirements along with the technologies, frameworks, libraries, and development tools used in the project.

Chapter 4: Proposed System Design

This chapter explains the overall design and architecture of the proposed system. It includes various UML diagrams such as the Class Diagram, Use Case Diagram, Activity Diagram, and Sequence Diagram. It also describes the system architecture, major modules, database design, invoice processing pipeline, AI/ML analysis pipeline, and interaction between the different components of CortexGrid.

Chapter 5: Implementation and Testing

This chapter explains the implementation of the project, including the frontend, backend, database, invoice upload and processing pipeline, PDF text extraction, OCR processing, background task processing, invoice data extraction, AI/ML analysis, Natural Language Processing, anomaly

detection, vendor matching, fraud analysis, risk scoring, and AI-assisted invoice verification. It also discusses the testing procedures used to verify the correctness, reliability, and performance of the system

Chapter 6: Conclusion and Future Scope

This chapter summarizes the overall project and discusses the benefits of the proposed system. It also presents possible future enhancements such as integration with enterprise accounting and ERP systems, advanced deep learning models for document understanding, multimodal invoice analysis, continuous model improvement using larger organizational datasets, real-time financial risk monitoring, and deployment of specialized domain-specific Large Language Models for invoice intelligence.

Chapter 7: References

This chapter contains the research papers, books, technical documentation, websites, and other resources referred to during the development of the project.

CHAPTER-2

LITERATURE SURVEY

Automated Invoice Processing and Document Extraction

The use of automated document processing has increased as organizations handle large volumes of invoices and other business documents. Invoice processing systems use document extraction techniques to identify important information such as invoice numbers, vendor details, dates, quantities, prices, and total amounts from digital documents. These approaches help reduce manual data entry and improve the efficiency of processing large numbers of invoices. PDF text extraction and document processing techniques are particularly useful for converting unstructured invoice documents into information that can be stored and processed electronically.

OCR-Based Processing of Scanned Invoices

Optical Character Recognition (OCR) techniques are widely used for extracting information from scanned documents and image-based files. In invoice processing, OCR can convert the text present in scanned invoice images into machine-readable text that can be further analyzed. This is important because invoices received by organizations may not always contain selectable text and may be available only as scanned documents. OCR-based processing therefore helps extend automated invoice processing to a wider range of document formats.

Automated Invoice Verification and Fraud Detection

Automated invoice verification systems can be used to compare extracted invoice information with stored business records and identify inconsistencies. Such systems can help detect issues including duplicate invoices, incorrect prices, quantity mismatches, and differences in vendor information. Rule-based verification provides a consistent method for checking invoice data against predefined conditions, while similarity-based techniques can help identify variations in vendor names.

Artificial Intelligence and Machine Learning for Invoice Intelligence

Artificial Intelligence and Machine Learning techniques can extend traditional invoice processing by enabling systems to analyze invoice information and identify patterns within financial data. Machine Learning-based approaches can be used for anomaly detection, classification, risk assessment, and

identification of potentially suspicious transactions. By analyzing multiple invoice attributes together, intelligent systems can identify unusual patterns that may not be easily detected through simple rule-based verification. This provides an additional layer of intelligence for automated invoice verification and financial control.

Natural Language Processing for Vendor and Invoice Information

Natural Language Processing (NLP) techniques can be applied to invoice-related text to understand and compare textual information. In invoice processing, NLP and text-similarity techniques can assist in identifying variations in vendor names and other textual fields. For example, differences in abbreviations, punctuation, spelling, or formatting may cause the same vendor to appear under different names. Intelligent text processing can help identify such variations and improve vendor matching and invoice verification.

Large Language Models for Intelligent Document Analysis

Large Language Models (LLMs) have introduced new possibilities for intelligent document analysis by providing capabilities for understanding and generating natural-language information. LLM-based systems can be integrated with document-processing pipelines to analyze extracted invoice information, identify relevant patterns, generate explanations, and assist in risk assessment. Such models can complement conventional rule-based and Machine Learning techniques by providing contextual analysis of structured and unstructured invoice information.

AI-Based Anomaly and Risk Analysis

AI-based anomaly detection can help identify invoices whose characteristics differ significantly from normal invoice patterns. Factors such as unusually high amounts, repeated submissions, unexpected vendor activity, inconsistent invoice information, and unusual transaction patterns can be considered during analysis. Combining verification results with AI/ML-based analysis can support the generation of a risk score and risk level for an invoice. This allows finance users to prioritize invoices that require additional review.

Integrated Invoice Intelligence Systems

Existing approaches demonstrate that invoice processing involves multiple stages including document extraction, OCR, data validation, verification, and financial analysis. However, these capabilities are often implemented as separate components. A more integrated approach can combine backend-based document processing and database management with AI/ML, NLP, and intelligent risk analysis in a

unified platform. Such integration can reduce manual effort while providing finance users with structured invoice information, verification results, and intelligent insights through a single system.

2.1 EXISTING WORK

In existing systems, invoice processing is generally performed through a combination of document extraction, OCR, rule-based validation, database management, and automated verification techniques. These systems can extract important invoice fields and compare them against predefined business rules or stored records. More recent approaches incorporate Machine Learning, NLP, and AI-based methods to improve anomaly detection, vendor matching, fraud analysis, and document understanding.

However, existing solutions may focus primarily on individual stages of invoice processing rather than providing a unified platform that combines document processing, backend automation, invoice verification, and AI/ML-based analysis. This creates an opportunity for an integrated invoice intelligence system that can process invoices, perform verification, apply intelligent analysis, and present the results through a centralized interface.

2.2 LIMITATIONS OF EXISTING WORK

Drawbacks of Existing System

- Many invoice processing systems depend heavily on manual verification for complex or inconsistent invoices.
- Different invoice formats may require separate document-processing and extraction techniques.
- Scanned invoices require OCR before their information can be processed automatically.
- Rule-based verification may not effectively identify complex or previously unseen invoice patterns.
- Variations in vendor names can make exact text-based vendor matching unreliable.
- Traditional systems may provide limited AI/ML-based anomaly and fraud analysis.
- Some solutions focus on document extraction without integrating intelligent financial risk assessment.
- Separate tools may be required for invoice processing, verification, vendor matching, and risk analysis.

• Limited integration between backend processing and AI/ML analysis can make the overall workflow less efficient.

• There is a need for a unified platform that combines **document intelligence, backend automation, NLP, AI/ML-based analysis, and invoice verification**.

CHAPTER-3

SOFTWARE AND HARDWARE SPECIFICATIONS

3.1 SOFTWARE REQUIREMENTS

- **Operating System**: Windows / Mac / Linux
- **Frontend Technologies**: HTML, CSS, JavaScript, React
- **Backend Technologies**: Python with Flask
- **Database**: PostgreSQL
- **ORM**: SQLAlchemy
- **Background Task Processing**: Celery
- **Message Broker**: Redis
- **PDF Processing**: PyMuPDF, pdfplumber
- **Authentication**: JWT-based Authentication
- **Development Tools**: Visual Studio Code
- **API Testing Tools**: Postman
- **Version Control**: Git and GitHub

3.2 FUNCTIONAL REQUIREMENTS

The **CortexGrid: Invoice Intelligence Platform** is designed as a web-based system for automating invoice ingestion, document processing, information extraction, verification, and financial risk analysis. The functional requirements of the system are described below.

User Authentication

This module manages secure access to the platform.

- The user can register an account.
- The user can log in using registered credentials.
- The system validates user credentials before granting access.
- The system uses secure authentication mechanisms.
- Authenticated users can access invoice and vendor management features.
- The user can securely log out of the application.

Invoice Upload

This module allows users to submit vendor invoices for processing.

- Users can upload invoices in PDF format.
- The system validates the uploaded file before accepting it.
- The system checks the file type and file size.
- Uploaded invoice files are stored securely.
- A corresponding invoice record is created in the database.
- The system assigns an appropriate processing status to each invoice.

PDF Text Extraction

This module extracts machine-readable text from digital PDF invoices.

- The system reads uploaded PDF documents.
- Text is extracted from the available pages.
- The extracted text is stored for further processing.
- The system supports invoices having different document layouts.
- Extracted information is passed to the invoice data extraction module.

OCR Processing

This module processes scanned and image-based invoices.

- The system identifies invoices where normal PDF text extraction is insufficient.
- PDF pages can be converted into images.
- OCR is applied to recognize text from invoice images.
- The extracted text is combined for further processing.
- OCR results are passed to the invoice information extraction module.

Invoice Information Extraction

This module converts extracted invoice content into structured information.

- The system extracts the invoice number.
- The system identifies the vendor name when available.
- The system extracts the invoice date.
- The system extracts the total invoice amount.
- The system identifies the currency.
- The system can extract invoice line-item information.
- Extracted information is stored in the database.
- The extracted information is displayed to authorized users.

Vendor Management

This module manages vendor information associated with invoices.

- Users can add vendor records.
- Users can view existing vendors.
- Users can update vendor information.
- Users can delete vendor records.
- The system maintains normalized vendor names.
- Vendor information can later be used for intelligent vendor matching.

Purchase Order Management

This module provides the foundation for comparing invoices with purchase orders.

- Users can create purchase-order records.
- Purchase orders can be associated with vendors.
- Purchase orders can contain multiple line items.
- Each line item can contain item name, quantity, and agreed price.

- Users can view and manage purchase-order information.
- Purchase-order information can later be compared with invoice data.

Invoice Verification
This module verifies invoice information against available business records.
- The system can compare invoice information with purchase-order information.
- Invoice prices can be compared with agreed purchase-order prices.
- Invoice quantities can be compared with purchase-order quantities.
- Differences between invoice and purchase-order information can be identified.
- Verification results can be stored for further analysis.

Duplicate Invoice Detection
This module identifies invoices that may have already been submitted.
- The system checks invoice numbers against previously processed invoices.
- Duplicate invoice numbers can be flagged.
- Identical uploaded documents can be identified using document-level comparison techniques.
- Duplicate invoices are retained for audit purposes rather than being silently deleted.
- The system provides an explanation when an invoice is identified as a possible duplicate.

Vendor Similarity Matching
This module handles variations in vendor names.
For example:
ABC Technologies Pvt Ltd
ABC Technology
ABC Tech Pvt. Ltd.
may refer to the same organization.
- The system normalizes vendor names.
- Similar vendor names can be compared.
- Similarity-based techniques can be used to identify potential vendor matches.
- Matching results can be used during invoice verification.

AI-Based Invoice Understanding
This module is planned as part of the intelligent processing layer of CortexGrid.
- AI/NLP techniques can be used to understand invoice content.
- The system can identify relevant entities and relationships within invoice text.
- AI-based extraction can improve handling of invoices with different layouts.
- Structured information generated by the AI layer can be passed to the verification engine.

Modern document-intelligence systems combine OCR with document understanding to extract fields and line items from invoices and other business documents.

Anomaly and Fraud Detection
This module provides intelligent analysis of invoice information.
- The system can identify unusual invoice amounts.
- The system can identify abnormal pricing or quantity patterns.
- Historical invoice information can be used to identify unusual vendor behavior.
- Machine-learning techniques can be incorporated for anomaly detection.
- Rule-based verification and ML-based analysis can be combined to improve fraud detection.

Risk Scoring
This module summarizes the results of invoice verification.

- The system evaluates identified invoice issues.
- Different verification findings can contribute to an invoice risk score.
- Invoices can be categorized according to their risk level.
- The system provides the reasons contributing to the risk score.
- High-risk invoices can be highlighted for further human review.

Background Invoice Processing
This module handles invoice processing asynchronously.

- Uploaded invoices are placed into a background processing queue.
- Redis is used as the message broker.
- Celery workers process invoice-processing tasks.
- PDF extraction, OCR, and data processing can occur in the background.
- The user interface remains responsive while processing is taking place.
- The invoice status is updated throughout the processing lifecycle.

Background processing is particularly useful for invoice workflows where document ingestion and extraction can involve multiple processing steps.

Dashboard and Reporting
This module provides a centralized interface for reviewing invoice activity.

- The dashboard displays the total number of invoices.
- The system displays processing and completed invoices.
- Failed invoice processing can be identified.
- Users can view extracted invoice information.
- Suspicious or high-risk invoices can be highlighted.
- Verification findings can be displayed along with their reasons.
- Users can review invoice information before taking further action.

Database Management
This module manages the application's persistent data.

- User information is stored in PostgreSQL.
- Vendor information is stored in PostgreSQL.
- Invoice information is stored in PostgreSQL.
- Purchase-order information is stored in PostgreSQL.
- Extracted invoice information is stored for verification.
- Verification and risk-analysis results can be stored for future reference.
- SQLAlchemy is used to manage database operations.

Error Handling
This module ensures reliable system operation.

- Invalid files are rejected.
- Invalid user inputs are handled.
- PDF processing failures are recorded.
- OCR failures are handled appropriately.
- Background task failures do not stop other invoice-processing tasks.
- Meaningful error messages are provided to users.
- The system prevents individual processing failures from crashing the complete application.

API Integration
This module manages communication between the frontend and backend.

- The React frontend communicates with the Flask backend through REST APIs.

- The backend validates incoming requests.
- The backend communicates with PostgreSQL for data storage and retrieval.
- Protected APIs require authentication.
- Structured responses are returned to the frontend.
- API errors are handled appropriately.

3.3 NON-FUNCTIONAL REQUIREMENTS

- The application should use modern web technologies such as **React, Flask, PostgreSQL, Celery, and Redis**.
- The system should provide efficient invoice processing and data extraction.
- The GUI should be simple, responsive, and user-friendly.
- **Usability:** The system should be easy for finance users to understand and operate.
- **Accuracy & Efficiency:** The system should extract invoice information accurately and process invoices with minimal delay.
- **Performance:** Background processing should ensure that invoice processing does not block the main web application.
- **Reliability:** The system should provide consistent processing results and handle errors without affecting the complete application.
- **Scalability:** The system should be capable of processing increasing numbers of invoices using asynchronous background tasks.
- **Security:** User credentials, authentication tokens, uploaded documents, and sensitive application configuration should be protected.
- **Data Integrity:** Invoice, vendor, purchase-order, and verification information should be stored consistently and reliably.
- **Maintainability:** The system should have a modular architecture that allows future AI/ML, fraud detection, and verification features to be added easily.
- **Explainability:** The system should provide understandable reasons for invoice flags and risk assessments rather than presenting only a final risk value.
- **Adaptability:** The system should support invoices with different formats, layouts, and document structures

3.4 HARDWARE REQUIREMENTS

- **Processor:** Intel i5 or above / Apple Silicon equivalent
- **RAM:** Minimum 8 GB
- **Storage:** Minimum 20 GB free space
- **Device:** Laptop / Desktop
- **Internet:** Required for development, API communication, and external services when applicable
- **GPU:** Not required for the current implementation, as PDF processing and OCR can be performed using CPU resources. If advanced ML/DL models are later deployed locally, a GPU may be used depending on the model requirements.

CHAPTER-4

PROPOSED SYSTEM DESIGN

4.1 PROPOSED SYSTEM

The The aim of the proposed system is to develop an intelligent and efficient **CortexGrid: Invoice Intelligence Platform** that automates the processing and verification of vendor invoices using document processing, backend automation, and AI/ML-based analysis. The system is designed to reduce the manual effort involved in reading invoices, extracting important information, checking invoice details, and identifying potentially suspicious transactions.

The proposed system allows finance users to upload vendor invoices in PDF format through a web-based interface. The uploaded documents are processed asynchronously in the background so that the main application remains responsive even when multiple or large invoices are submitted. The system extracts information from digital PDFs and uses OCR for scanned documents. The extracted information is then converted into structured invoice data for further processing and verification.

CortexGrid combines **deterministic backend verification with AI/ML-based intelligence**. Invoice information can be compared with purchase-order records to identify price and quantity mismatches, while duplicate detection and vendor similarity techniques can help identify suspicious or inconsistent invoices. AI/ML techniques can further support invoice understanding, vendor matching, anomaly detection, and fraud analysis. The results are presented through a centralized dashboard along with explanations of the identified issues.

Overall, the proposed system acts as an **intelligent invoice processing and verification platform** that helps organizations reduce manual work, improve invoice verification efficiency, identify suspicious transactions, and strengthen financial control.

Expected Advantages of Proposed System

The system is designed to be simple, efficient, and easy to use. It has the following features:

- Automates the initial processing of vendor invoices.
- Reduces manual data entry and verification effort.
- Extracts important information from PDF invoices.
- Supports scanned invoices through OCR processing.
- Processes invoices asynchronously using background tasks.
- Reduces application delays when multiple invoices are uploaded.

- Provides centralized invoice and vendor management.
- Compares invoice information with purchase-order records.
- Identifies price and quantity mismatches.
- Detects duplicate invoice submissions.
- Handles variations in vendor names using similarity-based matching.
- Supports AI/ML-based invoice understanding and anomaly detection.
- Generates risk scores for potentially suspicious invoices.
- Provides explanations for identified invoice issues.
- Provides a simple and interactive dashboard for finance users.
- Improves the efficiency and reliability of invoice verification.

Modules

Registered User Module
Invoice Upload Module
PDF Processing Module
OCR Processing Module
Invoice Information Extraction Module
Vendor Management Module
Purchase Order Management Module
Invoice Verification Module
Duplicate Invoice Detection Module
Vendor Similarity Matching Module
AI/ML Analysis Module
Fraud Detection & Risk Scoring Module
Background Processing Module
Database Management Module
Dashboard & Reporting Module

The Registered User Module includes:

- User Registration/Login: Allows users to create accounts and securely access the platform.
- Upload Invoice: Allows users to submit vendor invoices in PDF format.
- View Invoices: Allows users to view previously uploaded invoices and their processing status.
- View Invoice Details: Displays extracted invoice information and processing results.
- Manage Vendors: Allows authorized users to add, view, update, and delete vendor information.
- Manage Purchase Orders: Allows users to manage purchase-order records and associated line items.
- Review Verification Results: Allows users to examine invoice verification findings.
- View Risk Information: Allows users to view the risk level and reasons associated with suspicious invoices.

Invoice Upload Module

This module allows users to upload vendor invoice documents through the web application. The system validates the uploaded PDF, stores the document securely, creates an invoice record, and places the invoice into the background processing queue.

PDF Processing Module

This module extracts machine-readable text from uploaded PDF documents. PyMuPDF and pdfplumber can be used to process digital PDF invoices and obtain the text required for further invoice information extraction.

OCR Processing Module

This module handles scanned or image-based invoices where normal PDF text extraction is insufficient. The PDF pages are converted into images and processed using OCR technology to obtain machine-readable text.

Invoice Information Extraction Module

This module converts the extracted invoice content into structured information such as invoice number, vendor name, invoice date, currency, total amount, and invoice line items. Basic extraction methods are used in the current implementation, while AI/NLP-based extraction can be incorporated into the intelligent processing layer.

Vendor Management Module

This module manages vendor information associated with invoices and purchase orders. It stores vendor details and maintains normalized vendor names. The information provides the foundation for future similarity-based vendor matching.

Purchase Order Management Module

This module manages purchase orders and their associated items. Each purchase order can contain information such as the vendor, item name, quantity, and agreed price. These records provide the reference data required for invoice verification.

Invoice Verification Module

This module compares extracted invoice information with purchase-order information. It can identify differences in item prices, quantities, and other relevant invoice details. The verification results are used to determine whether an invoice requires further review.

Duplicate Invoice Detection Module

This module identifies invoices that may have already been submitted. It can compare invoice numbers and document information with previously processed invoices and flag potential duplicate submissions for review.

Vendor Similarity Matching Module

This module handles variations in vendor names across different documents. Text normalization and similarity-based techniques can be used to determine whether differently written vendor names are likely to represent the same organization.

AI/ML Analysis Module

This module forms the intelligent component of CortexGrid. AI, NLP, and machine-learning techniques can be used to improve invoice information extraction, understand invoice content, identify vendor similarities, and detect unusual invoice patterns. The module is designed to work alongside deterministic backend verification rather than replacing it.

Fraud Detection & Risk Scoring Module

This module analyzes the results obtained from invoice verification and intelligent analysis. Price mismatches, quantity discrepancies, duplicate submissions, unusual patterns, and other findings can contribute to the invoice's risk assessment. The system generates a risk score and provides the reasons contributing to the identified risk.

Background Processing Module

This module manages asynchronous invoice processing using Celery and Redis. Uploaded invoices are placed into a background queue and processed by Celery workers. PDF extraction, OCR, and invoice processing can therefore take place without blocking the main web application.

Database Management Module

This module manages persistent application data using PostgreSQL and SQLAlchemy. It stores user, vendor, invoice, purchase-order, extracted-data, and verification information and provides reliable data retrieval for the application.

Dashboard & Reporting Module

This module provides a centralized interface for finance users to monitor invoice processing and verification activities. It displays invoice status, extracted information, verification findings, risk levels, and other relevant information required for reviewing potentially suspicious invoices.

4.2 CLASS DIAGRAM

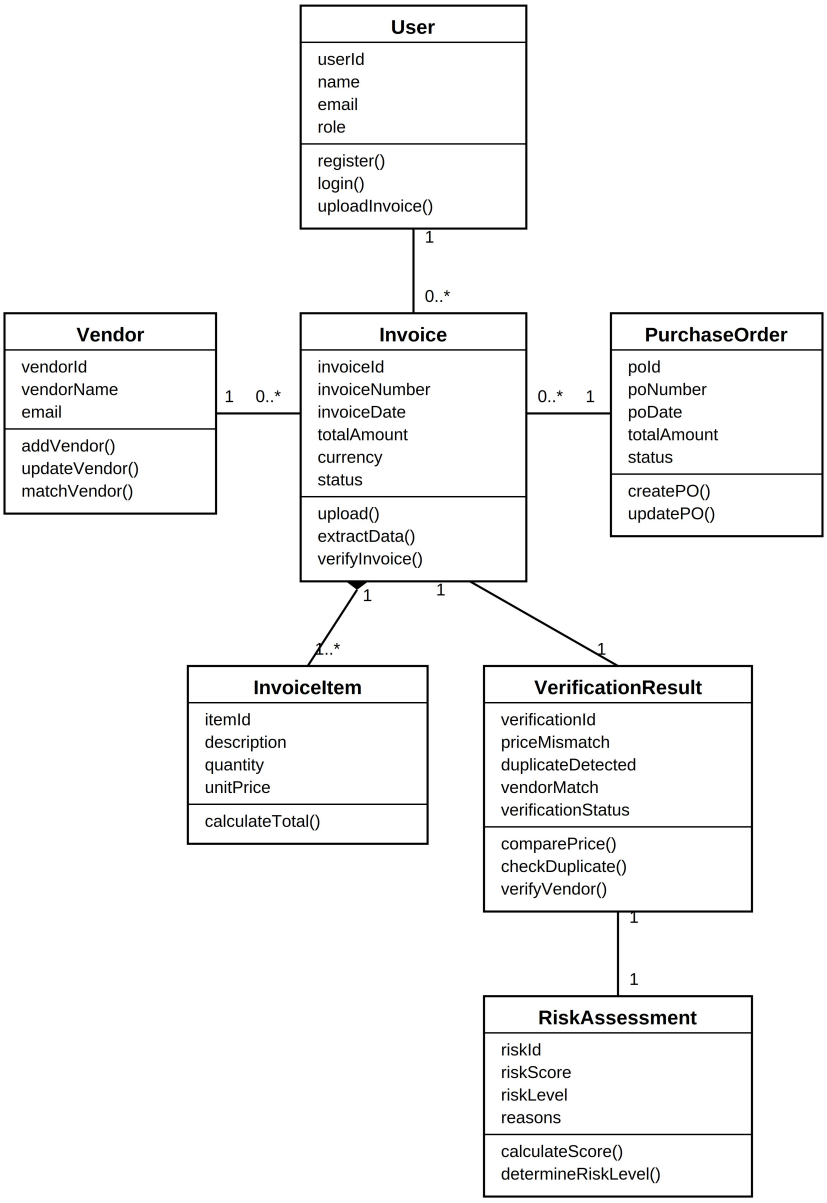


Fig-1: Class Diagram

The class diagram of **CortexGrid: Invoice Intelligence Platform** represents the main entities involved in invoice processing, vendor management, purchase-order management, and verification. The major classes include **User, Vendor, Invoice, InvoiceItem, PurchaseOrder, PurchaseOrderItem, VerificationResult, and RiskAssessment**.

The **User** class manages authentication and user-related information. The **Vendor** class stores vendor details and is associated with multiple invoices and purchase orders. The **Invoice** class contains invoice information such as invoice number, date, total amount, currency, processing status, and extracted data. The **InvoiceItem** class represents individual items present in an invoice.

The **PurchaseOrder** class stores purchase-order information and is associated with **PurchaseOrderItem** classes containing item names, quantities, and agreed prices. The **VerificationResult** class stores the results of invoice verification, such as price mismatches, quantity mismatches, and duplicate detection. The **RiskAssessment** class represents the risk level and reasons identified during invoice analysis.

The relationships between these classes represent the flow of information from invoice submission and extraction to verification and risk assessment.

4.3 USE CASE DIAGRAM

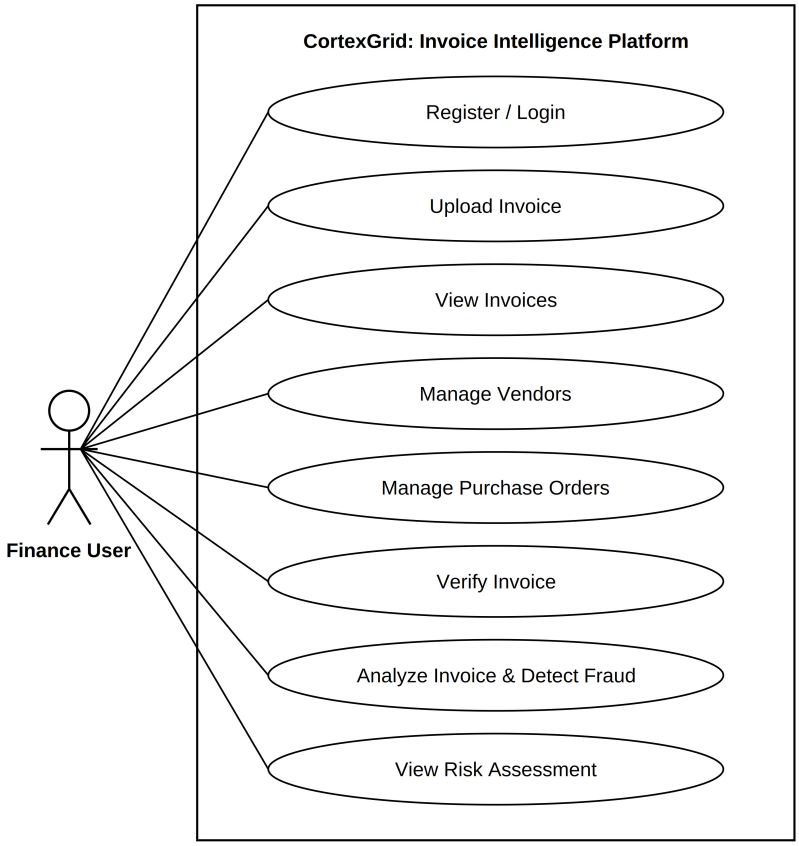


Fig- 2: Use Case Diagram.

The The use case diagram represents the interactions between the users and **CortexGrid: Invoice Intelligence Platform**. The primary user interacts with the system to **register, log in, upload invoices, view invoices, manage vendors, manage purchase orders, view extracted information, and review verification results**.

The system processes uploaded invoices through PDF extraction and OCR and provides the extracted information to the user. The user can also view invoice processing status and verification results.

The verification-related use cases include **matching invoices with purchase orders, checking price and**

quantity differences, detecting duplicate invoices, matching vendor information, and viewing risk assessments. The diagram represents how the user interacts with the different modules of CortexGrid to process and verify invoices.

4.4 ACTIVITY DIAGRAM

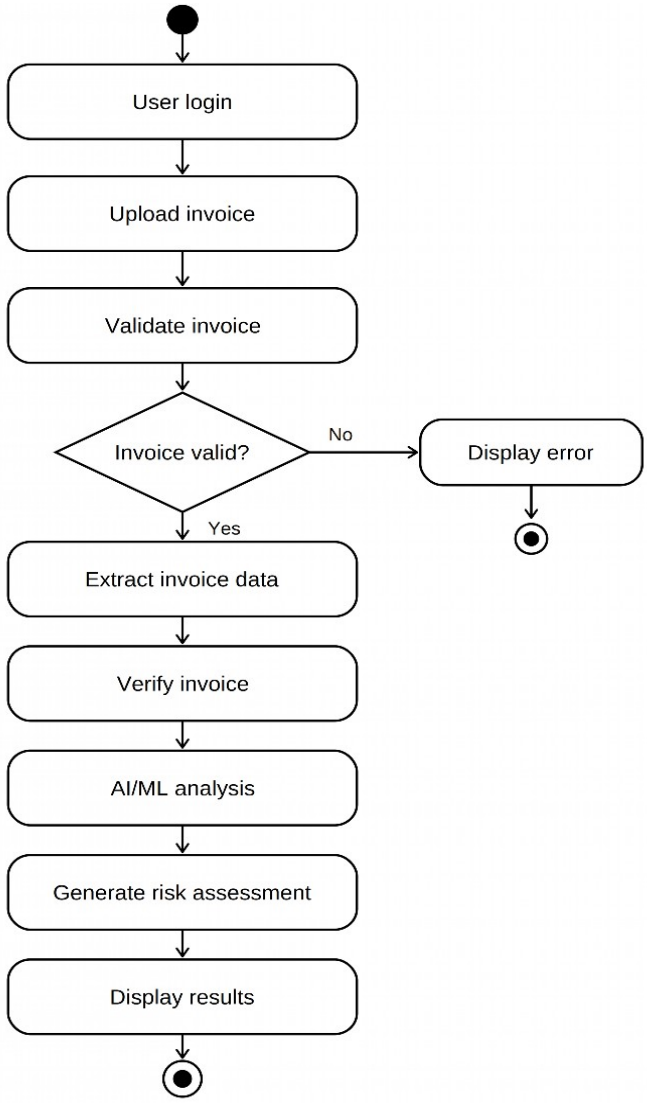


Fig 3: User Activity Diagram

The activity diagram represents the overall workflow of **CortexGrid: Invoice Intelligence Platform**. The user first logs into the system and uploads a vendor invoice in PDF format. The system validates the uploaded document and places it into the background processing queue.

The invoice is then processed by the background worker. The system attempts to extract text from the PDF and uses OCR when the document is scanned or when sufficient text cannot be extracted normally. The extracted information is then converted into structured invoice data.

After extraction, the invoice information can be associated with the corresponding vendor and purchase order. The system performs verification checks such as **price comparison, quantity comparison, duplicate detection, and vendor matching**. The results are then analyzed to determine the invoice risk level and are displayed to the user through the dashboard.

Overall, the activity diagram shows the complete workflow from **invoice upload to document processing, information extraction, verification, and risk assessment**.

4.5 SEQUENCE DIAGRAM

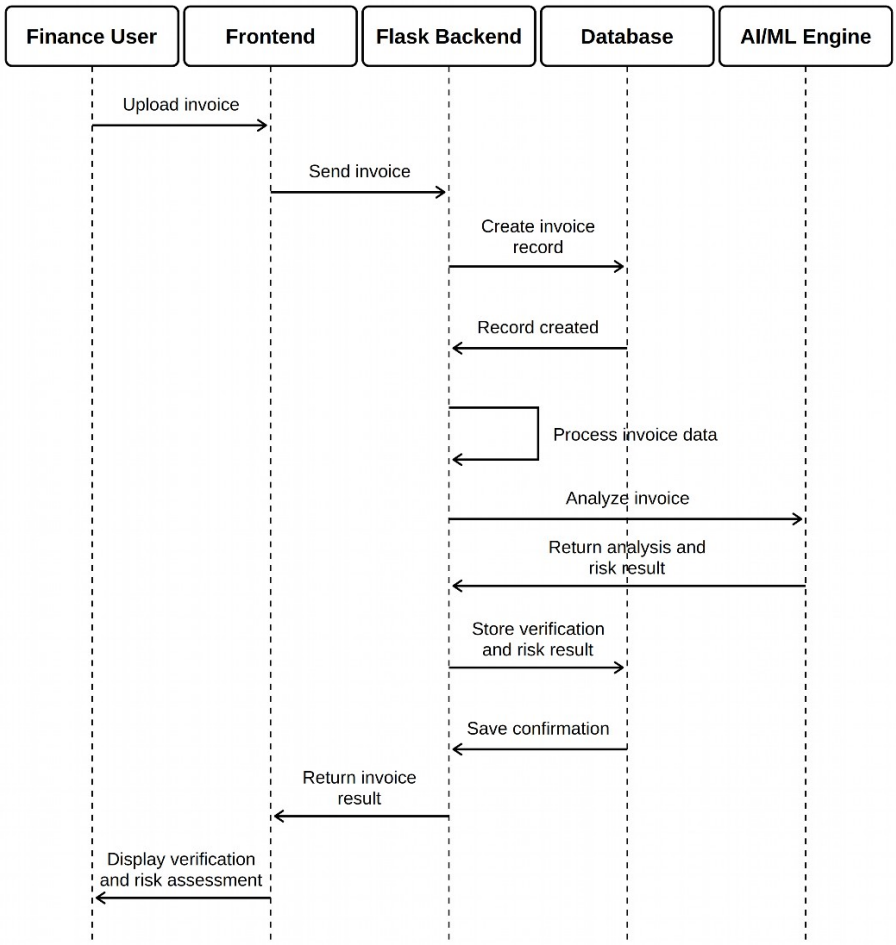


Fig 4: Sequence Diagram

The sequence diagram represents the interaction between the **User, Frontend, Flask Backend, Database, Redis, Celery Worker, and Invoice Processing Services** in CortexGrid.

The user uploads an invoice through the frontend. The frontend sends the request to the Flask backend, which validates the uploaded file, stores the invoice information in the database, and places a processing task into the Celery queue using Redis. The backend immediately returns the processing status to the frontend.

The Celery worker receives the task and processes the invoice in the background. It performs PDF text extraction and uses OCR when required. The extracted information is then passed to the invoice parsing and processing services. The processed invoice information and status are stored in the database.

The frontend periodically retrieves the invoice status and information from the backend and displays the result to the user. In the complete system, the processed invoice data is then passed to verification and AI/ML analysis components for further evaluation.

Overall, the sequence diagram illustrates the communication flow between the **user interface, backend services, asynchronous processing system, database, and document-processing components**.

4.6 SYSTEM ARCHITECTURE

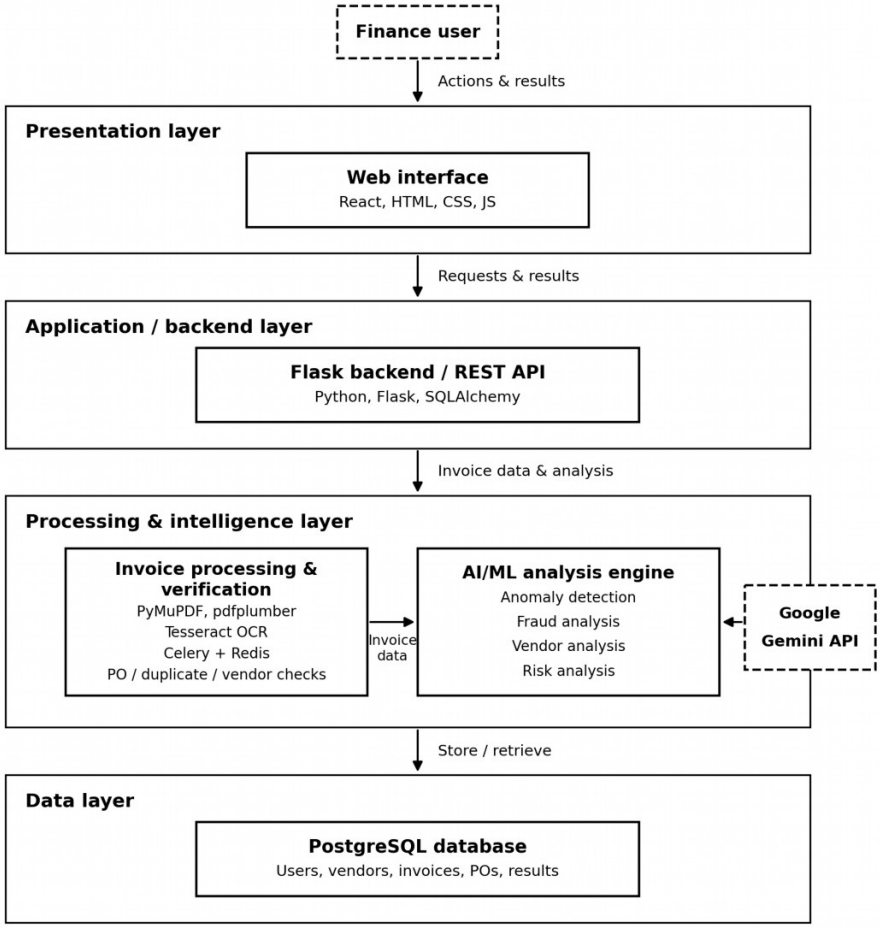


Fig-5: System Architecture.

The system architecture of **CortexGrid: Invoice Intelligence Platform** follows a modular full-stack architecture consisting of **Frontend, Backend, Database, Background Processing, Document Processing, and AI/ML Intelligence** components.

The **Frontend** is developed using React, Vite, and Tailwind CSS and provides the interface through which users can log in, upload invoices, manage vendors, view purchase orders, and review invoice-processing results.

The **Backend** is developed using Python Flask and manages authentication, REST APIs, business logic, invoice management, vendor management, purchase-order management, and communication between the different components of the system.

The **Database** uses PostgreSQL with SQLAlchemy to store user, vendor, invoice, invoice-item, purchase-order, purchase-order-item, extraction, verification, and risk-related information.

The **Background Processing Layer** uses Celery and Redis to process invoices asynchronously. This allows PDF extraction, OCR, and other computational tasks to execute in the background without blocking the main web application.

The **Document Processing Layer** uses PyMuPDF and pdfplumber for PDF text extraction and Tesseract OCR for scanned or image-based invoices. The extracted information is converted into structured invoice data.

The **AI/ML Intelligence Layer** forms the intelligent component of CortexGrid. It can support invoice information extraction, NLP-based document understanding, vendor similarity matching, anomaly detection, and fraud analysis. The AI/ML layer works together with deterministic backend verification to provide reliable and explainable invoice analysis.

The **Verification and Risk Assessment Layer** compares invoice information with purchase-order records, identifies duplicate invoices and discrepancies, and combines verification findings with intelligent analysis to generate a risk assessment.

4.7 TECHNOLOGY DESCRIPTION

Python

Python is the primary programming language used for backend development and document-processing operations. It provides libraries required for PDF processing, OCR integration, database communication, background task processing, and implementation of AI/ML-related components.

Flask

Flask is used as the backend web framework of CortexGrid. It handles REST API requests, authentication, business logic, invoice management, vendor management, and communication between the frontend, database, document-processing services, and background workers.

React.js, HTML5, CSS3 and JavaScript

React.js, HTML5, CSS3, and JavaScript are used to develop the frontend of CortexGrid. HTML5 provides the basic structure of the web pages, CSS3 is used for styling and layout, and JavaScript provides interactivity. React.js is used to organize the interface into reusable components and manage different application views. The frontend provides interfaces for user authentication, invoice upload, vendor management, invoice monitoring, and viewing invoice-processing results.

PostgreSQL

PostgreSQL is used as the relational database management system. It stores structured information such as user details, vendor records, invoices, invoice items, purchase orders, and verification results. The relational structure helps maintain consistent relationships between different entities of the system.

SQLAlchemy

SQLAlchemy is used as the Object Relational Mapper (ORM) between the Flask application and PostgreSQL. It allows database tables to be represented as Python models and simplifies database operations such as creating, retrieving, updating, and deleting records.

PyMuPDF and pdfplumber

PyMuPDF and pdfplumber are used for processing PDF invoice documents. They extract machine-readable text from digital PDF files, which is then passed to the invoice information extraction and parsing modules.

Tesseract OCR

Tesseract OCR is used to extract text from scanned or image-based invoices. When normal PDF text extraction is insufficient, the invoice pages can be processed using OCR to convert the visible text into machine-readable information.

Google Gemini API

The Google Gemini API provides the generative AI capabilities planned for the intelligent processing layer of CortexGrid. It can be used to understand unstructured invoice content, assist with information extraction, and generate meaningful explanations of invoice analysis results. The AI component works together with deterministic backend verification rather than replacing database-based calculations.

Celery

Celery is used for asynchronous background task processing. Invoice processing tasks such as PDF extraction, OCR, and other computational operations can be executed by background workers instead of blocking the main Flask application. This allows the system to handle multiple invoice-processing requests more efficiently.

Redis

Redis is used as the message broker for Celery. When an invoice is uploaded, the processing task is placed into Redis and retrieved by a Celery worker. This enables invoice processing to take place asynchronously in the background.

AI/ML and NLP Technologies

AI, Machine Learning, and Natural Language Processing technologies form the intelligent analysis layer of CortexGrid. These technologies can be used for invoice information extraction, understanding unstructured invoice text, vendor similarity matching, anomaly detection, and fraud analysis. The AI/ML layer is designed to complement deterministic verification rules and database-based comparisons.

PDF Processing and Document Intelligence

The PDF processing component converts unstructured invoice documents into information that can be stored and analyzed by the system. It combines PDF text extraction and OCR to support both digital and scanned invoices. The extracted information provides the foundation for invoice verification and AI/ML-based analysis.

Postman

Postman is used for testing and verifying the REST APIs developed using Flask. It can be used to test authentication, invoice, vendor, purchase-order, and other backend endpoints by sending requests and examining their responses.

python-dotenv

The python-dotenv library is used to manage environment variables securely. Sensitive configuration information

such as database credentials, JWT secrets, Redis configuration, and API keys can be stored in environment files instead of being directly written into the source code.

Visual Studio Code

Visual Studio Code is used as the primary Integrated Development Environment (IDE) for developing CortexGrid. It provides features for writing, debugging, testing, and managing the Python backend, React frontend, database-related code, and other project files.

Git and GitHub

Git is used for version control and tracking changes made during development. GitHub is used to maintain the project repository, collaborate between team members, and manage different versions of the source code.

Docker, Gunicorn and Nginx

Docker can be used to containerize the different components of CortexGrid and provide a consistent deployment environment. Gunicorn can be used as a production WSGI server for the Flask application, while Nginx can act as a reverse proxy and web server for handling incoming requests and serving the application efficiently.

REFERENCES

[1] **Priyanka Malla (2025).** *A Scalable Enterprise Framework for AI-Driven Invoice Processing Using Document Intelligence.* International Journal of AI, BigData, Computational and Management Studies, 6(4), 181–186. This paper presents an enterprise invoice-processing framework that combines OCR, NLP, and machine learning for automated invoice processing. It is directly relevant to CortexGrid because it addresses document extraction, automation, and reducing manual invoice verification effort. Available from: https://ijaibdcms.org/index.php/ijaibdcms/article/view/345

[2] **Yangxin Xu et al. (2020).** *LayoutLM: Pre-training of Text and Layout for Document Image Understanding.* Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD 2020), pp. 1192–1200. This research introduces a transformer-based approach that jointly uses text and document layout information for document understanding and information extraction. It is relevant to CortexGrid because invoices often contain structured fields and tables whose meaning depends on both the text and its position within the document. Available from: https://arxiv.org/abs/1912.13318

[3] **Michail Dadopoulos and Stratos Moschidis (2026).** *Beyond Fuzzy Matching: A Dual-Augmentation RAG System for Robust Product Reconciliation in Accounting.* Journal of Risk and Financial Management, 19(6), 402. This paper investigates AI-assisted entity reconciliation in accounting data under OCR noise and variations in naming. It is relevant to CortexGrid for vendor and item matching, where the same vendor or product may appear under different names or formats in invoices and related records. Available from: https://ideas.repec.org/a/gam/jjrfmx/v19y2026i6p402-d1956251.html